

Budget App

Architecture, layers & data flow

Local-first budgeting with **Hive**, cloud backup via **Supabase**, and **Cubit** state in the UI.

Tech stack

Area	Choice
UI & state	Flutter, <code>flutter_bloc</code> (Cubit)
Local persistence	Hive (<code>hive_ce</code>), typed models + adapters
Remote / auth	Supabase (<code>supabase_flutter</code>)
Composition	Manual wiring in <code>main.dart</code> (no <code>get_it</code> in repo)
Domain rules	Entities + repository interfaces; some shared <code>core/</code> helpers

Feature module layout

Single feature folder: `lib/features/budget/`

```
features/budget/  
├── domain/           ← entities, abstract repositories  
├── data/             ← models, datasources, repository impls  
└── presentation/    ← pages, cubits, shared widgets
```

Cross-cutting: `lib/core/` (e.g. `InsufficientBalanceException`, `budget_defaults`, `id_generator`).

The three layers (dependency direction)



- **Domain:** `entities/`, `repository/*.dart` — no Flutter imports in entities/repos.
- **Data:** `models/` (DTOs), `datasource/`, `*_repo_imp.dart` — maps **Model** ↔ **Entity**, talks to Hive & Supabase.
- **Presentation:** `pages/`, `*/cubit/` — calls repositories through Cubits; maps errors to user-visible strings.

Composition root: `main.dart`

1. **Bootstrap:** `.env` → `Supabase.initialize`, `Hive.initFlutter`, register adapters.
2. **Auth:** `AuthRepoImp` + global `AuthCubit` + `MaterialApp` → `_AppAuthGate`.
3. **Session:** On authenticated user → `_AuthenticatedSessionScope(userId)` builds:
 - `LocalDatasource(userId)` — Hive box names **scoped per user**
 - `RemoteDatasource(SupabaseClient)`
 - `AccountRepoImp`, `CategoryRepoImp`, `TransactionRepoImp`
 - `MultiBlocProvider`: `AccountCubit`, `TransactionCubit`, `CategoryCubit`
→ `MainTabPage`

`ValueKey(userId)` ensures a **fresh widget subtree** when the signed-in user changes.

Data sources

LocalDataSource

- Hive boxes: **accounts**, **categories**, **transactions** (suffix from `userId`).
- CRUD for models; deleting a category nulls `categoryId` on related transactions locally.

RemoteDataSource

- Tables: `accounts` , `categories` , `transactions` .
- Uses `currentUserId` from Supabase session; filters with `user_id` .
- **Upsert / delete / list** — JSON ↔ `*Model` (`fromJson` / manual mapping for transactions).

Repository pattern (implemented)

Interface (domain)	Implementation	Collaborators
AuthRepository	AuthRepoImp	Supabase auth only
AccountRepository	AccountRepoImp	Local + optional remote
CategoryRepository	CategoryRepoImp	Local + optional remote
TransactionRepository	TransactionRepoImp	Local + AccountRepository + optional remote

Note: TransactionRepoImp takes AccountRepository to update balances atomically with new transactions.

Read path (local-first, remote hydrate)

Typical flow — `AccountRepoImp.getAllAccounts` /
`TransactionRepoImp.getTransactions` :

1. Read from Hive
2. If empty AND remote available AND user logged in:
 fetch from Supabase → persist each row to Hive → re-read local
3. Map Model → Entity (transactions enrich with category from local if present)
4. Return to Cubit → emit success state

Offline-friendly: **UI reads cache first**; cloud fills gaps when local is empty.

Write path (local + best-effort cloud)

1. Validate / compute (e.g. new balance for expenses).
2. `_local.save*` (source of truth for UX latency).
3. `try { await _remote?.upsert* } catch (_) {}` — failures **do not roll back** local write in current code.

Auth uses Supabase only (no Hive session mirror beyond app state).

Transaction creation flow (domain rules)

```
TransactionCubit.createTransaction(...)
  → TransactionRepository.createTransaction(entity)
    → get account; compute balance after income/expense
    → if balance < 0 → InsufficientBalanceException
    → AccountRepository.updateAccount(balance)
    → save TransactionModel locally
    → remote upsert (swallowed on error)
```

Cubit maps `InsufficientBalanceException` to a stable user message.

Presentation state

- **Auth:** `AuthCubit` / `AuthState` — gate for `_AuthenticatedSessionScope`.
- **Budget:** `AccountCubit`, `CategoryCubit`, `TransactionCubit` with loading / success / failure substates (`TransactionCubit` clears transient failures when list still valid).

UI: `MaterialApp` today (project rules mention Cupertino as a target style for future alignment).

Supabase schema (conceptual)

- Rows keyed by `id` with `user_id` for RLS-style scoping in app code.
- Migrations under `supabase/migrations/` define tables and evolution (e.g. text PKs migration in repo history).

Summary

Concern	Approach
User isolation	Hive box suffix per <code>userId</code> ; remote queries filter <code>user_id</code>
Sync model	Local-first reads; hydrate from remote when local empty; writes local + best-effort remote
Boundaries	Domain repos; data owns models & I/O; presentation owns Cubits & copy
Critical invariant	Non-negative account balance enforced in <code>TransactionRepoImp</code>

End

Deck source: `docs/budget_app_architecture_slides.md`

Export with [Marp](#) / VS Code Marp extension / `@masaki39/marp-mcp`.